

MAXMYCLOUD SECRETS & ENVIRONMENT VARIABLES

Container-image deployment reference

Reference for populating your own secret store (Vault, AWS SSM, Kubernetes Secrets, or equivalent) when deploying the MaxMyCloud container in your AWS account.

Values are grouped by **who provides them** so you know which fields you fill in vs. which come from MaxMyCloud.

1. MaxMyCloud provides these values

ENV VAR	VALUE FOR YOUR INSTALL
<code>NUXT_PUBLIC_TENANT_CLIENT_ID</code>	MaxMyCloud will send per-customer, e.g. <code>aig-poc</code>
<code>NUXT_PUBLIC_TENANT_DISPLAY_NAME</code>	MaxMyCloud will send, e.g. <code>AIG</code> — displayed on the login screen and inside authenticator apps for MFA
Container image	<code>ghcr.io/maxmycloud/maxmycloud-ui:<tag></code> — MaxMyCloud will pin you to a specific release

2. You generate these values yourselves

Cryptographic keys and your tenant-admin password hash — generate on a trusted workstation, store in your secret manager, never reuse across environments. Do not share back to MaxMyCloud.

ENV VAR	HOW TO GENERATE	PURPOSE
<code>NUXT_JWT_SECRET</code>	<code>openssl rand -base64 64</code>	Signs session tokens
<code>NUXT_MFA_SECRET_KEY</code>	<code>openssl rand -base64 32</code>	AES-256 key encrypting stored TOTP secrets
<code>passwordEncryptionKey</code>	<code>openssl rand -hex 32</code> — must be hex (64 chars = 32 bytes)	AES-256 key encrypting stored Snowflake credentials. Name is lowercase (no <code>NUXT_</code> prefix) to match the container's expectation.
<code>NUXT_CUSTOMER_ADMIN_BCRYPT_HASH</code>	see one-liner below	bcrypt hash of the password YOUR tenant admin will use for first sign-in. Password stays with your admin; only the hash goes into your secret store.

How to generate the bcrypt hash on a workstation

```
# One-time – install bcryptjs if it isn't already available on the workstation
npm install --no-save bcryptjs

# Have your tenant admin pick a strong password (12+ chars, high entropy),
# then generate the hash. Password NEVER leaves this machine – only the hash does.
node -e "console.log(require('bcryptjs').hashSync('<your-admin-password>', 12))"
# Output looks like: $2b$12$AbC.../HdEfG...
```

Paste the hash (starts with `$2b$12$`) into your secret store as `NUXT_CUSTOMER_ADMIN_BCRYPT_HASH`. Do NOT paste the plaintext password anywhere.

3. You supply these values from your own environment

ENV VAR	WHERE IT COMES FROM
<code>MONGODB_URI</code>	Connection string to your Mongo-compatible database. See § 3a below if you are using AWS DocumentDB.
<code>MONGODB_DB</code>	Database name — pick anything sensible, e.g. <code>maxmycloud</code>
<code>NUXT_CUSTOMER_ADMIN_EMAIL</code>	The email address of your tenant admin — the person who will sign in first to configure Snowflake SSO and take over day-to-day management.

3a. AWS DocumentDB users — TLS + connection-string requirements

DocumentDB requires TLS with the Amazon RDS certificate authority, and rejects the retryable-write behavior that most MongoDB drivers use by default. Your `MONGODB_URI` must include **all three** of the following parameters:

PARAM	VALUE	WHY
<code>tls</code>	<code>true</code>	DocumentDB rejects plaintext connections
<code>tlsCAFile</code>	<code>/etc/ssl/certs/global-bundle.pem</code>	The Amazon RDS CA bundle. This file is baked into the MaxMyCloud container image — no action required from you
<code>retryWrites</code>	<code>false</code>	DocumentDB does not implement retryable writes; the driver default (<code>true</code>) errors out on connect

Example connection string:

```
mongodb://<user>:<password>@<cluster-endpoint>.docdb.amazonaws.com:27017/?
tls=true&tlsCAFile=/etc/ssl/certs/global-bundle.pem&retryWrites=false
```

URL-encode any special characters in the password (`@` , `:` , `/` , `?` , `#` , `%` , etc.). Do **not** use `tlsAllowInvalidCertificates=true` — it disables server-identity verification and is not appropriate for a production install.

4. Explicitly set to disable optional features (recommended for POC)

ENV VAR	VALUE	WHY
<code>NEXT_AI_FEATURES_DISABLED</code>	<code>true</code>	Disables AI features (chatbot, AI-generated health-check narratives). No Amazon Bedrock IAM permissions needed on the task role. Can be turned on later.

5. Not required for the POC (leave unset)

ENV VAR	REASON
<code>NEXT_LLM_PROVIDER</code> , <code>NEXT_BEDROCK_*</code> , <code>AWS_REGION</code> (for Bedrock)	AI is off for the POC; no LLM calls attempted
SES / SMTP env vars	POC has no email features (password-reset emails, scheduled health-check email digests)

ENV VAR	REASON
<code>NUXT_PUBLIC_APP_URL</code>	Only used server-side to build URLs embedded in outbound emails; unused for the POC since email is off
<code>MAXMYCLOUD_REPLAY_DIR</code>	Only needed if you mount a shared snapshot volume; POC doesn't need it

Complete summary — what gets set in your secret store

For the standard POC install, exactly **10 env vars** need to be populated:

```

MONGODB_URI           = <your Mongo connection string>
MONGODB_DB            = maxmycloud
NUXT_JWT_SECRET        = <64 random bytes, base64>
NUXT_MFA_SECRET_KEY    = <32 random bytes, base64>
passwordEncryptionKey  = <32 random bytes, HEX – openssl rand -hex 32>
NUXT_CUSTOMER_ADMIN_EMAIL = <your tenant admin's email>
NUXT_CUSTOMER_ADMIN_BCRYPT_HASH = <bcrypt hash of their chosen password>
NUXT_PUBLIC_TENANT_CLIENT_ID   = <MaxMyCloud will send>
NUXT_PUBLIC_TENANT_DISPLAY_NAME = <MaxMyCloud will send>
NUXT_AI_FEATURES_DISABLED     = true

```

Plus the container image tag MaxMyCloud pins you to. That's the entire configuration surface.

First-time setup — how it works

Once your container is running and the database is reachable:

1. **Your tenant admin signs in** at your internal application URL using the email you set as `NUXT_CUSTOMER_ADMIN_EMAIL` and the password whose hash you stored as `NUXT_CUSTOMER_ADMIN_BCRYPT_HASH`.
2. **They enroll TOTP MFA** on first sign-in (scan the QR code with an authenticator app — Google Authenticator, Microsoft Authenticator, Authy, 1Password, etc.). This is required for all Superadmin accounts.
3. **They configure the Snowflake SSO integration** — with MaxMyCloud support watching on a screenshare and directing the OAuth Client ID / Client Secret / redirect-URI values.
4. **They create their first Snowflake account connection** in the app.
5. From then on, your tenant admin (and any additional users they invite) signs in via **Snowflake SSO**. The initial email/password path is only needed for that first sign-in.

Post-install: what you can rotate yourselves

- **Any env var** — update in your secret manager, roll a container restart
- **Users' passwords** — Settings → Users → Reset password (for password-based users)
- **Snowflake credentials** — Settings → Accounts → Manage
- **Encryption keys** — rotate `NUXT_JWT_SECRET` , `NUXT_MFA_SECRET_KEY` , `passwordEncryptionKey` on a schedule that matches your security policy. Note: rotating `NUXT_MFA_SECRET_KEY` or `passwordEncryptionKey` invalidates existing encrypted values in the database, so plan accordingly.

Contact

Send the two values from § 3 back to MaxMyCloud when your infrastructure is provisioned. Everything in § 2 stays inside your environment.

MaxMyCloud — Snowflake cost optimization platform